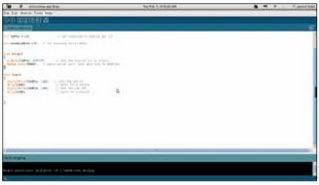

Figure 1: Arduino IDE

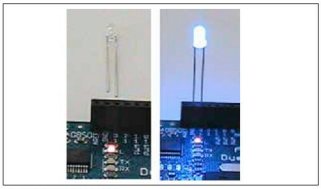

Figure 2: Arduino toggles LED

variable, and the last line displays the value of the variable. If everything went well, the output should be the same string written to the serial port—"*Jayneil Dalal\n*".

Arduino

I had made a mention of Arduino in the first article. Now, let me explain how to use it, with a few examples. Let's use the Arduino IDE (Figure 1) to write code. You can install it via the Ubuntu Software Centre by just searching for 'Arduino'. After the installation, launch it. Connect your Arduino to your computer via the USB cable. Then select the target board—go to *Tools>Board* and make the choice. Next, configure the serial port via *Tools>Serial* and select the available port. Connect the anode of the LED to pin 13, and the cathode to the GND (ground) pin. Now, we are all set to write our first bit of Arduino code (*Arduino_LED.pde*, shown below) to get an LED to flicker:

```
int ledPin = 13;                  // LED connected to digital
pin 13

void setup() {
  pinMode(ledPin, OUTPUT);      // sets the digital pin as
output
  Serial.begin(9600);   // opens serial port, sets data rate
to 9600 bps
}
void loop() {
  digitalWrite(ledPin, LOW);   // sets the LED on
  delay(1000);                 // waits for a second
  digitalWrite(ledPin, LOW);    // sets the LED off
  delay(1000);                 // waits for a second
}
```

Now click the *Verify* button to check for errors in the program, and then click the *Upload* button to dump the code to the Arduino. If everything has gone well, you will see the LED toggle 'on' and 'off' (Figure 2).

I have included comments in the code for further explanation. There are basically two modules in Arduino, which are *setup* and *loop*. The first runs once, while the latter runs infinite times. In the fifth line, I defined a variable for the pin number to connect the anode of the LED. Then in *setup()*, I have declared that I will use that pin as an output pin. I then initialised serial communication with a baud rate of 9600. Now look at the *loop()* module. Here, I first set the value of pin 13 as 'low' (0, because the pin is digital). This will turn off the LED. After a delay of 1 second, I set the pin value to 'high' (1), which will turn on the LED, followed by another 1 second delay. This *loop()* will run forever, hence the LED will continue to turn on and off.

Now, I hope most of you have read the second article in this series, and are familiar with OpenCV by now. So here's the OpenCV code (*BuildingES.py*) that will detect faces from a given image, and if the operation is successful, will trigger the Arduino via Pyserial:

```
#!/usr/bin/python

import cv  #import the openCV lib to python
import serial #import the pyserial module

#Module -1: Image Processing
hc = cv.Load("/home/jayneil/haarcascade_frontalface_default.
xml")
img = cv.LoadImage("/home/jayneil/beautiful-faces.jpg", 0)
faces = cv.HaarDetectObjects(img, hc, cv.CreateMemStorage())
a=1
print(faces)
for (x, y, w, h), n in faces:
    cv.Rectangle(img, (x, y), (x+w, y+h), 255)
cv.SaveImage("faces_detected.jpg", img)
dst=cv.LoadImage("faces_detected.jpg")
cv.Namedwindow("Face Detected", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("Face Detected", dst)
cv.WaitKey(5000)
cv.DestroyWindow("Face Detected")

#Module -2: Trigger Pyserial
```